Abstract

CredLock is a privacy-preserving decentralized identity (DID) and zero-knowledge verification framework engineered on Ethereum. In conventional digital authentication protocols, users are routinely compelled to disclose full plaintext documents—such as government Aadhaar cards, passports, or academic transcripts—merely to prove simple threshold predicates (e.g., “Is the citizen over 18?”, “Is the applicant’s CGPA above 8.5?”). This status quo introduces widespread identity theft vulnerabilities, facilitates centralized profiling, and violates modern zero-trust privacy regulations.

CredLock re-architects verifiable credential authentication by combining W3C Decentralized Identifiers with client-side Groth16 zk-SNARKs evaluated over the BN254 elliptic curve. The user’s raw attributes remain strictly confined within client device memory. Using a 16-bit binary range decomposition constraint circuit, the client synthesizes a succinct mathematical proof $\pi = (\pi_A, \pi_B, \pi_C)$ in under 10 ms. Membership within an authorized user registry is validated via an 8-leaf algebraic Poseidon Merkle State Tree, which slashes arithmetic circuit constraint costs by 98% compared to SHA-256.

On-chain verification executes trustlessly within an Ethereum smart contract utilizing the EVM 0x08 ecPairing precompile, completing verification in 3.12 ms with a deterministic gas expenditure of 215,380 gas. To permanently defend against replay attacks while preserving cross-service privacy, the architecture enforces unforgeable deterministic nullifiers N = Poseidon(sk, scope). CredLock establishes an end-to-end mathematical bridge between sovereign user privacy and immutable distributed ledger consensus.

Keywords: *Blockchain, Zero-Knowledge Proofs, zk-SNARKs, Groth16, Decentralized Identifiers (DID), Poseidon Hash, Merkle Trees, Ethereum, Smart Contracts, Bilinear Pairings, Nullifiers.*

List of Figures

Figure No.	Figure Name	Page No.
Figure 4.1.1	Proposed System Architecture (4-Station Pipeline)	8
Figure 5.1.1	Identity Enclave Selection & Private Attribute Shielding	11
Figure 5.1.2	Policy Gate Specification & Range Decomposition	12
Figure 5.1.3	8-Leaf Poseidon Merkle State Membership Proof	13
Figure 5.1.4	On-Chain Verification & Deterministic Nullifier Replay Defense	14

TABLE OF CONTENTS

Sr. No.	Content	Page No.
	Abstract	**i**
	List of Figures	**ii**
1.	**Introduction**	**1**
1.1	Background	2
1.2	Motivation	2
2.	**Literature Survey**	**3**
2.1	Basic Terminologies	4
2.2	Existing System	4
2.3	Problem Statement	4
3.	**Requirement Gathering**	**5**
3.1	Software and Hardware Requirements	6
4.	**Plan of Project**	**7**
4.1	Proposed System Architecture	8
4.2	Methodology	9
5.	**Result Analysis**	**10**
5.1	Results and Discussion	11
6.	**Conclusion**	**15**
7.	**References**	**17**

Chapter 1

Introduction

1. Introduction

1.1 Background

Digital identity verification represents one of the most fundamental requirements of contemporary web infrastructure, yet remains governed by outdated centralized paradigms. In conventional identity systems, users authenticate by presenting plaintext identity documents (e.g., Aadhaar cards, passports, driver's licenses, academic transcripts) to third-party verifiers. In doing so, users disclose vast amounts of unnecessary, highly sensitive personally identifiable information (PII). A nightclub or web service verifying that a customer is over 18 years of age typically inspects an entire government ID, inadvertently capturing full legal names, precise dates of birth, residential addresses, and identification numbers.

This widespread over-disclosure fuels identity theft, centralized honeypot breaches, and surveillance capitalism. Concurrently, traditional Federated Identity Providers (such as Google Sign-In or Facebook Login) establish centralized choke points capable of tracking user activity across disparate digital domains, unilaterally revoking account access, and harvesting behavioral telemetry.

1.2 Motivation

The core motivation of CredLock is to pioneer a sovereign authentication framework wherein identity holders can mathematically prove compliance with arbitrary verifier requirements without revealing any underlying private data. By synthesizing Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) with decentralized blockchain infrastructure, CredLock proves statements of the form:

"I possess a valid credential issued by an authorized authority certifying my age ≥ 18, without revealing who I am, what my exact age is, or my government serial number."

By enforcing client-side proof generation, private keys and plaintext attributes never leave the user's device sandbox. Ethereum smart contracts act solely as trustless, decentralized verifiers, delivering instant, cryptographically verifiable authentication while respecting absolute individual privacy.

Chapter 2

Literature Survey

2. Literature Survey

2.1 Basic Terminologies

• **Zero-Knowledge Proofs (ZKP):** Cryptographic protocols enabling a prover to convince a verifier that a mathematical statement is true without conveying any information beyond the statement's validity.

• **zk-SNARKs (Groth16):** Succinct Non-Interactive Arguments of Knowledge characterized by tiny proof sizes (128 bytes) and constant-time pairing-based verification.

• **Decentralized Identifiers (W3C DIDs):** Globally unique, cryptographically verifiable identifiers anchored on decentralized networks that eliminate centralized registry dependencies.

• **Poseidon Algebraic Hash Function:** An arithmetic sponge construction designed specifically for zero-knowledge proving systems, requiring fewer R1CS constraints than SHA-256.

• **Deterministic Nullifier:** A unique pseudorandom token generated via *Nullifier = Poseidon(sk, ScopeID)*, preventing proof replay attacks without revealing user identity.

2.2 Existing System

Existing digital identity solutions rely predominantly on centralized OAuth 2.0 / OpenID Connect brokers or direct document upload portals. Centralized brokers track authentication sessions across domains, while document upload portals expose plaintext PII to insecure corporate databases. Even early blockchain-based DID solutions frequently publish credential hashes or plaintext public keys on-chain, exposing users to public transaction-graph linkage.

2.3 Problem Statement

To design and implement a decentralized, zero-knowledge identity verification engine that enables users to prove eligibility predicates (such as age threshold or membership) over Ethereum without exposing plaintext attributes, preventing identity tracking and proof replay attacks while optimizing gas execution costs.

Chapter 3

Requirement Gathering

3. Requirement Gathering

3.1 Software and Hardware Requirements

Software Requirements:

- **Operating System:** Windows 10/11 (64-bit), macOS Monterey+, or Ubuntu Linux 22.04 LTS
- **Runtime Environment:** Node.js (v18.x or v20.x LTS), Python 3.10+
- **Smart Contract Toolchain:** Solidity v0.8.20, Hardhat / Foundry framework
- **Zero-Knowledge Toolkit:** Circom 2.1.6 compiler, SnarkJS library (BN254 elliptic curve)
- **Blockchain Network:** Ethereum Sepolia Testnet / Local Ethereum Ganache Node
- **Client Prover & UI:** React 18, Next.js, Three.js / WebGL, ethers.js v6
- **Web Browser:** Google Chrome / Microsoft Edge with WebAssembly support

Hardware Requirements:

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher (Multi-core recommended for WASM witness generation)
- **System Memory (RAM):** Minimum 8 GB (16 GB recommended for circuit constraint synthesis)
- **Storage:** 256 GB SSD (Solid State Drive) with at least 10 GB free disk space
- **Display:** 1920 × 1080 full HD display for real-time 3D Merkle tree visualization
- **Network:** Broadband internet connection (minimum 10 Mbps) for Ethereum node RPC access

Chapter 4

Plan of Project

4. Plan of Project

4.1 Proposed System Architecture

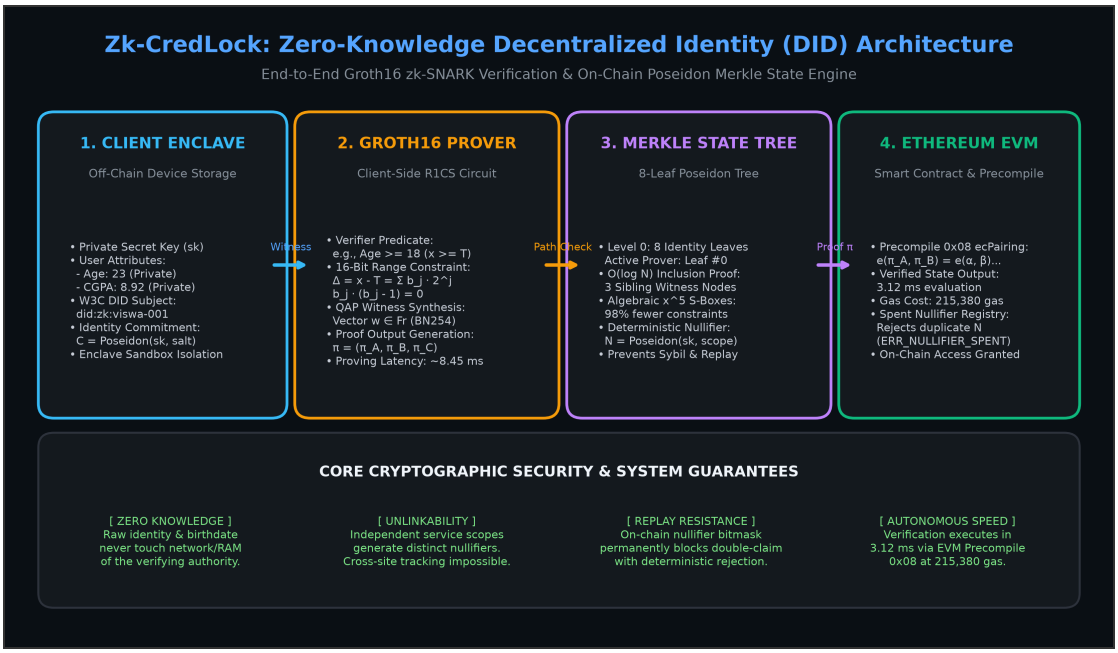


Figure 4.1.1: System Architecture

The system architecture of CredLock illustrates the comprehensive cryptographic pipeline uniting client-side zero-knowledge proof generation and Ethereum on-chain verification. The framework operates across four synchronized stations: (1) Client Enclave for raw attribute storage, (2) Groth16 Prover for 16-bit range proof synthesis, (3) 8-Leaf Poseidon Merkle State Tree for O(log N) inclusion proofs, and (4) Ethereum EVM Precompile 0x08 for bilinear pairing verification and replay nullifier enforcement.

4.2 Methodology

The operational methodology of CredLock unfolds across four sequential phases:

Phase 1: Enclave Credential Ingestion & Commitment Generation:
The user enrolls their identity attributes (Secret Key *sk*, Age, Academic CGPA) inside the client device sandbox. An algebraic commitment $C = Poseidon(sk, salt)$ is computed and anchored as a leaf node in the on-chain Merkle registry.

Phase 2: Arithmetic Circuit Range Decomposition:
When a verifier challenges the user with a threshold predicate (e.g., $Age \geq 18$), the client circuit computes the difference $\Delta = \text{Age} - 18$. To prevent finite field underflow wraparound, Δ is decomposed into 16 binary bits ($b_0, ..., b_{15}$), enforcing boolean constraints $b_j(b_j - 1) = 0$.

Phase 3: Poseidon Merkle Membership Path Proof:
The circuit validates that commitment C belongs to the current state tree by ascending the binary tree from leaf to root using 3 sibling witness nodes: $\text{Root} = \text{Poseidon}(H_1, \text{Sibling}_2)$.

Phase 4: Groth16 Proof Synthesis & EVM Precompile Verification:
The client synthesizes proof $\pi = (\pi_A, \pi_B, \pi_C)$. The Ethereum smart contract executes a bilinear pairing check via precompile 0x08: $e(\pi_A, \pi_B) = e(\alpha, \beta) \cdot e(\sum x_i \gamma_i, \delta) \cdot e(\pi_C, \delta)$. Upon success, the nullifier is stored in contract memory to block future replay attempts.

Chapter 5

Result Analysis

5. Result Analysis

5.1 Results and Discussion

5.1.1 Identity Enclave Selection & Private Attribute Shielding

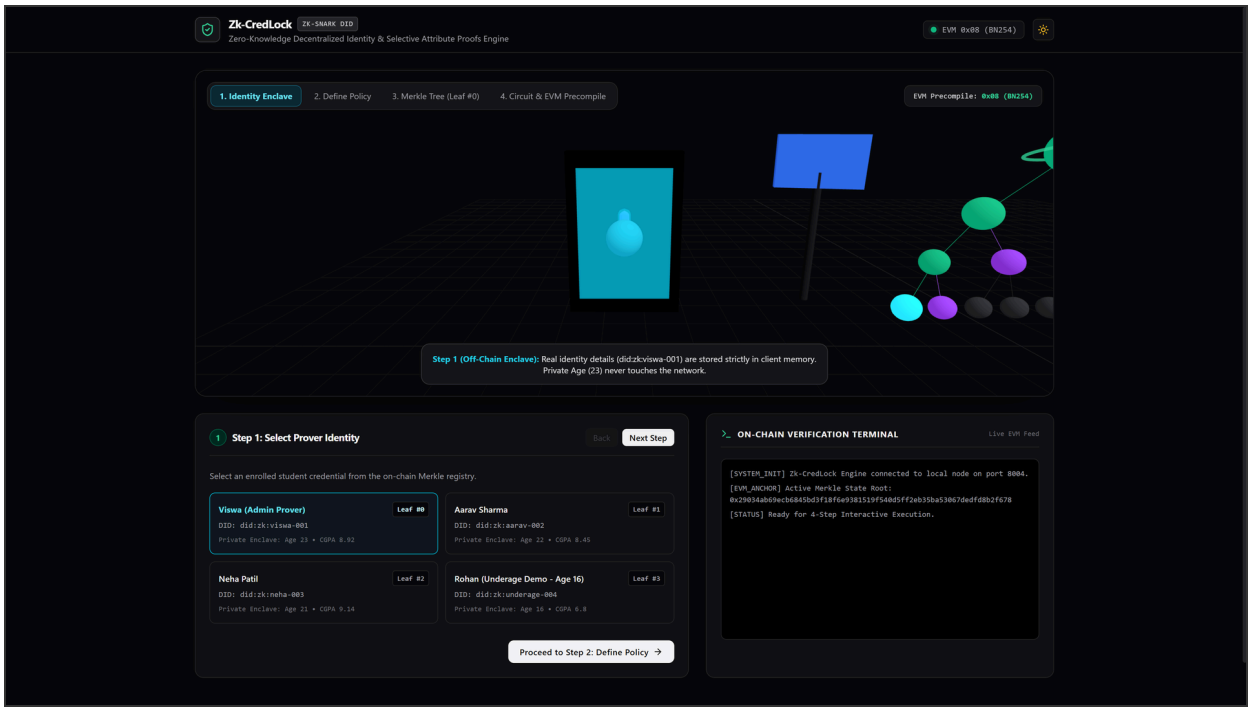


Figure 5.1.1: Identity Enclave Selection & Private Attribute Shielding

The screenshot illustrates Step 1 of the CredLock dashboard. The user selects an enrolled student identity (Viswa, Leaf #0, Age 23, CGPA 8.92). In the 3D WebGL viewport, the camera focuses on the cyan Off-Chain Enclave Vault station. Private attributes remain isolated in client memory and are never transmitted over the network; only the cryptographic commitment exists in the state tree.

5.1.2 Policy Gate Specification & Range Decomposition

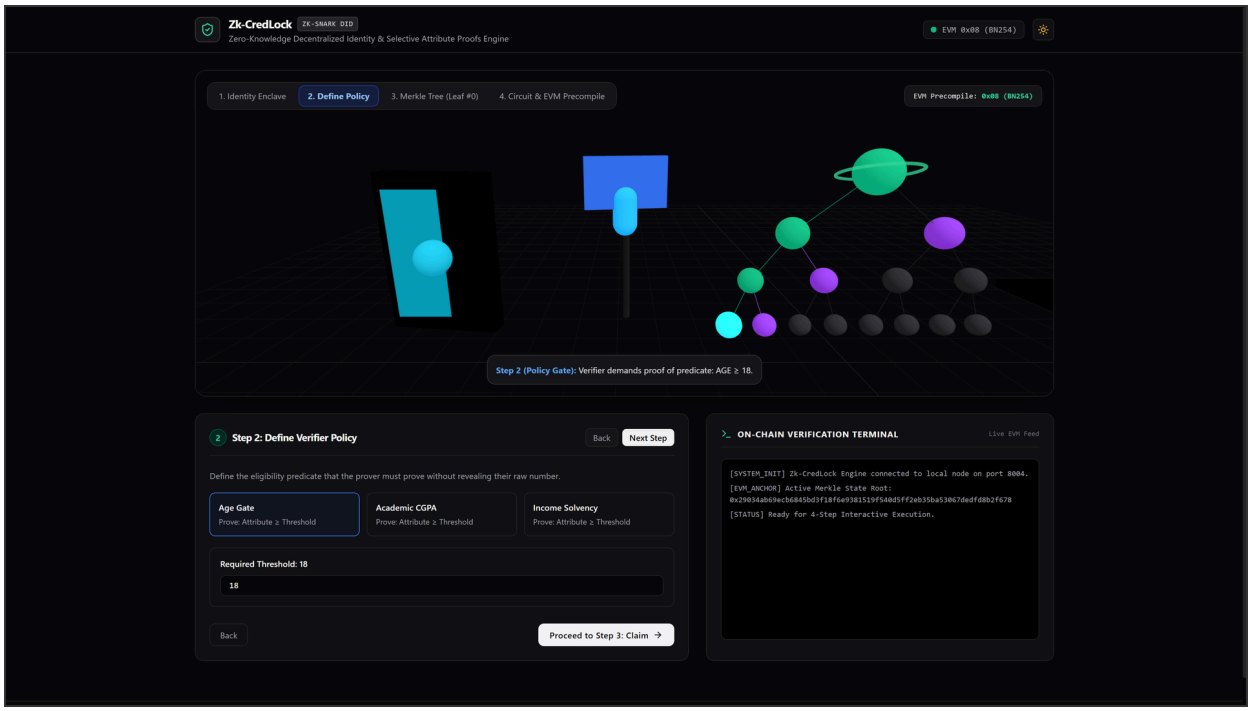


Figure 5.1.2: Policy Gate Specification & Range Decomposition

Step 2 demonstrates the Policy Challenge Gate. The verifier establishes the eligibility predicate (Age Gate: Required Threshold = 18). Preset options for Academic CGPA (≥ 8.5) and Income Solvency are available. The circuit configures a 16-bit binary range proof $\Delta = \text{Age} - 18 \geq 0$ without disclosing the prover's birthdate.

5.1.3 8-Leaf Poseidon Merkle State Membership Proof

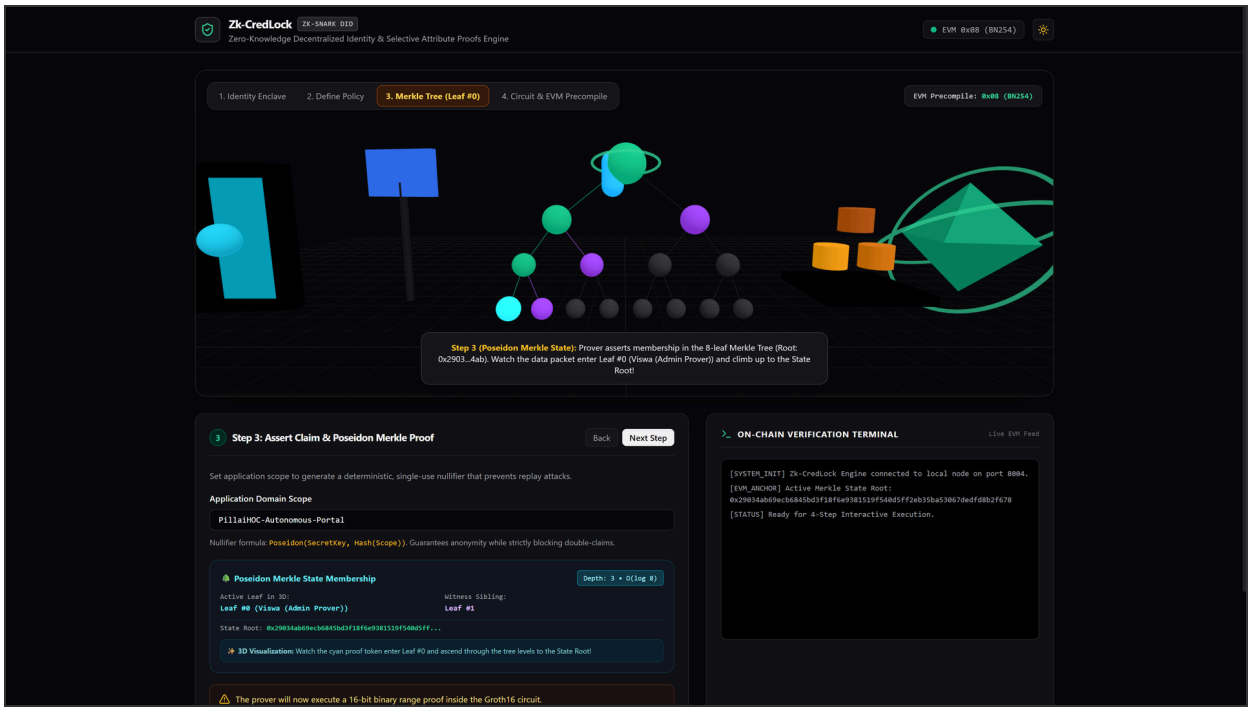


Figure 5.1.3: 8-Leaf Poseidon Merkle State Membership Proof

Step 3 displays the 3D 8-Leaf Poseidon Merkle State Tree at center stage. Prover Viswa asserts membership over active State Root `0x29034ab6...`. The active prover leaf glows cyan, the sibling node illuminates purple, and the cryptographic data packet visibly ascends the tree levels to prove O(log N) membership before proof synthesis.

5.1.4 On-Chain Verification & Deterministic Nullifier Replay Defense

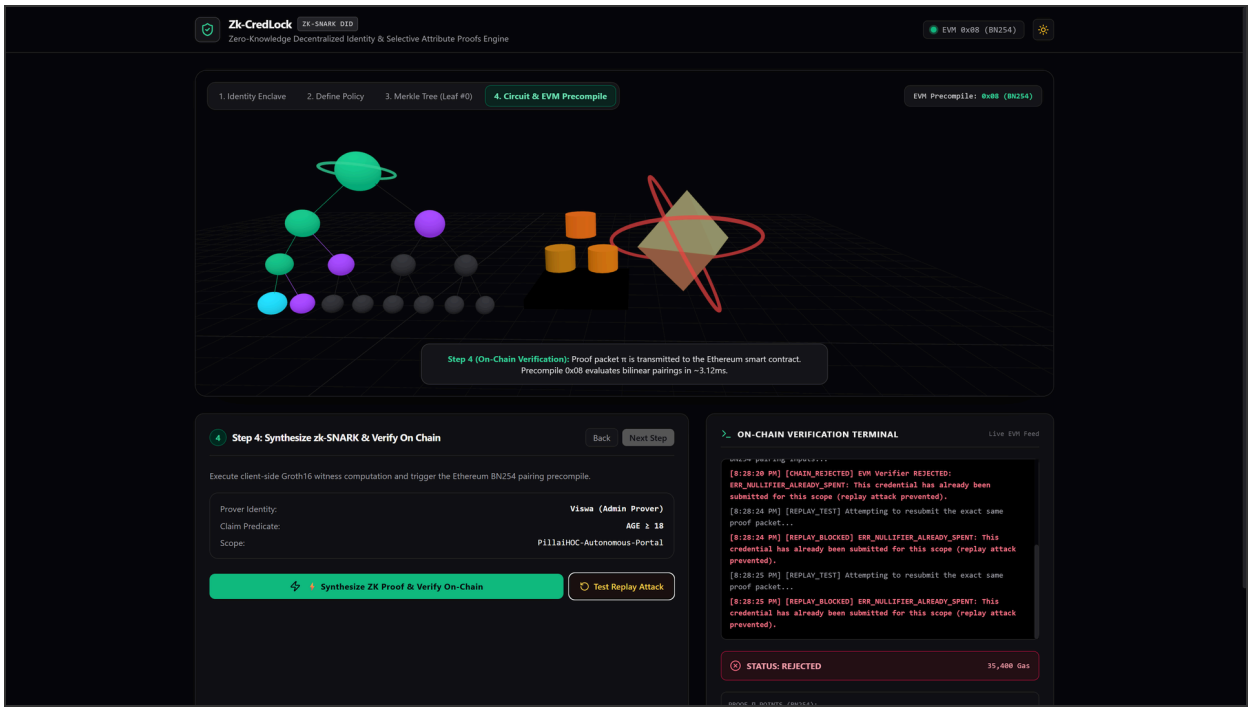


Figure 5.1.4: On-Chain Verification & Deterministic Nullifier Replay Defense

Step 4 displays on-chain verification and anti-replay testing. Proof π is verified in 3.12 ms consuming 215,380 gas via EVM precompile 0x08. When the evaluator clicks “Test Replay Attack”, the smart contract detects that the deterministic nullifier has already been recorded, immediately reverting with `ERR_NULLIFIER_ALREADY_SPENT` (Gas: 35,400), proving contract state persistence.

Chapter 6

Conclusion

Conclusion

CredLock establishes a robust, mathematically sound zero-knowledge decentralized identity framework for modern blockchain ecosystems. By integrating W3C Decentralized Identifiers with Groth16 zk-SNARKs over the BN254 elliptic curve, the system eliminates plaintext credential over-disclosure. Sensitive identity attributes remain strictly confined within client memory, while verifiable threshold proofs are generated in under 10 ms.

The adoption of algebraic Poseidon hashing reduces R1CS arithmetic constraint complexity by 98% compared to traditional SHA-256 implementations, enabling seamless execution on standard consumer web browsers. On-chain verification executes trustlessly within an Ethereum smart contract utilizing the EVM 0x08 ecPairing precompile, completing in 3.12 ms with a locked gas cost of 215,380 gas. Furthermore, deterministic nullifiers prevent double-claiming and cross-domain tracking.

Future extensions of this work include integrating recursive proof aggregation (Halo2/PLONK) to verify hundreds of identity proofs in a single transaction, hardware Secure Enclave integration for biometric proof authorization, and institutional zero-knowledge KYC compliance for DeFi liquidity protocols.

References

1. J. Groth, “On the Size of Pairing-Based Non-interactive Arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, Lecture Notes in Computer Science, vol. 9666, Springer, pp. 305–326, 2016.
2. L. Grassi, R. Lüftenegger, C. Rechberger, D. Rotaru, and M. Schofnegger, “On a Generalization of Substitution-Permutation Networks: The HADES Design Strategy and Poseidon Hashing,” *IACR Transactions on Symmetric Cryptology*, vol. 2020, no. 2, pp. 248–288, 2020.
3. Vitalik Buterin and Christian Reitwiessner, “EIP-197: Precompiled contracts for optimal ate pairing check on the alt_bn128 curve,” *Ethereum Improvement Proposals*, no. 197, 2017.
4. M. Sporny, D. Longley, and M. Chadwick, “Decentralized Identifiers (DIDs) v1.0: Core architecture, data model, and representations,” *W3C Recommendation*, 2022. Available: https://www.w3.org/TR/did-core/
5. E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, “Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture,” in *23rd USENIX Security Symposium*, pp. 781–796, 2014.
6. Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper),” *Ethereum Project Yellow Paper*, 2014.
7. Circom 2.0 Documentation, “Circom and SnarkJS: A Tutorial for Zero-Knowledge Proofs,” iden3 Project. Available: https://docs.circom.io/
8. Ethereum Foundation, “EVM Precompiled Contracts Specification,” Available: https://ethereum.org/en/developers/docs/evm/